S.K.A.K. Madhushan

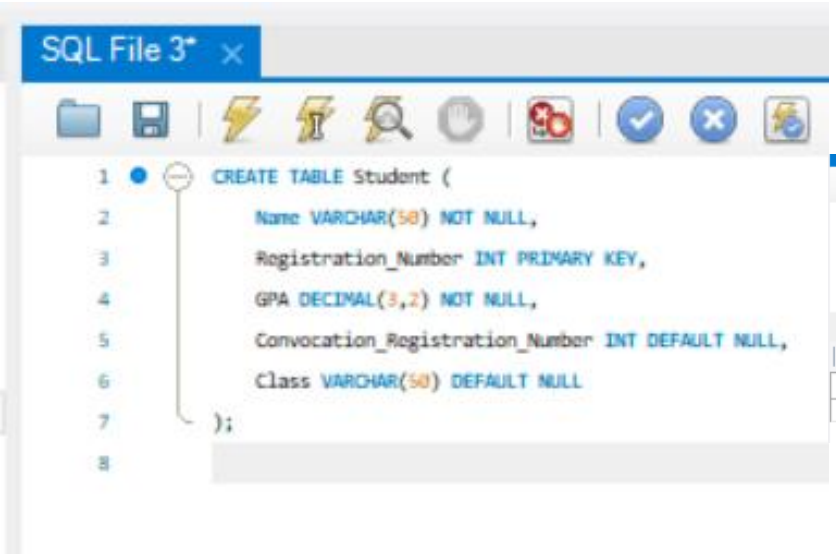
E/21/245

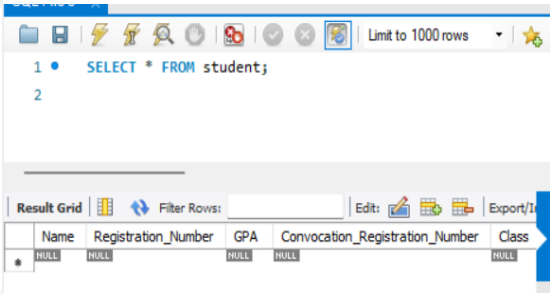
CO226 – DATABASE SYSTEMS LAB 05

Following table contains grade records of some final year students of a faculty.

Name	Registration Number	GPA	Convocation Registration Number	Class
Akila	154	3.80		
Ishta	149	3.55		
Chamara	327	3.25		
Haritha	118	3.75		
Saadia	147	4.00		
Thamish	379	3.95		
Nimuthu	398	3.25		
Ridma	155	3.85		
Ravindu	224	3.10		
Vilakshan	373	3.70		

1. Create a table named Student to insert the above data. Determine a suitable primary key for this table. Populate your table with these data.





```sql
INSERT INTO Student (Name, Registration_Number, GPA) VALUES
('Akila', 154, 3.80),
('Ishta', 149, 3.55),
('Chamara', 327, 3.25),
('Haritha', 118, 3.75),
('Saadia', 147, 4.00),
('Thamish', 379, 3.95),
('Nimuthu', 398, 3.25),
('Ridma', 155, 3.85),
('Ravindu', 224, 3.10),
('Vilakshan', 373, 3.70);
```

```sql
select * from student;
```

Result Grid

Name	Registration_Number	GPA	Convocation_Registration_Number	Class
Haritha	118	3.75	NULL	NULL
Saadia	147	4.00	NULL	NULL
Ishta	149	3.55	NULL	NULL
Akila	154	3.80	NULL	NULL
Ridma	155	3.85	NULL	NULL
Ravindu	224	3.10	NULL	NULL
Chamara	327	3.25	NULL	NULL
Vilakshan	373	3.70	NULL	NULL
Thamish	379	3.95	NULL	NULL
Nimuthu	398	3.25	NULL	NULL

2. Create another table named Convocation to store the details about the registration to the convocation. Table should have fields to hold the Last Name, Address, Age, Registration Number and Convocation Registration Number of each student. The Convocation Registration Number is given when a student registers for the convocation. (read Lab task 4. Before doing the current task) Determine a suitable primary key and a foreign key for this table.

```sql
-- task 2
CREATE TABLE Convocation (
    Last_Name VARCHAR(50) NOT NULL,
    Address VARCHAR(100) NOT NULL,
    Age INT NOT NULL,
    Registration_Number INT NOT NULL,
    Convocation_Registration_Number INT PRIMARY KEY,
    FOREIGN KEY (Registration_Number) REFERENCES Student(Registration_Number)
);
```

```sql
select * from convocation;
```

Result Grid

Last_Name	Address	Age	Registration_Number	Convocation_Registration_Number
NULL	NULL	NULL	NULL	NULL

3. Create a stored procedure to store the details about the students as one student at a time in the Convocation table when they register for the convocation. (In the registration a unique convocation registration number will be given to each student and at the same time Convocation Registration Number column in the Student table should be updated by inserting relevant convocation registration numbers for that student.)

```
-- task 3

DELIMITER //
CREATE PROCEDURE RegisterStudent(
    IN p_last_name VARCHAR(50),
    IN p_address VARCHAR(100),
    IN p_age INT,
    IN p_registration_number INT,
    IN p_convocation_reg_number INT
)
BEGIN
    -- Insert into Convocation table
    INSERT INTO Convocation (Last_Name, Address, Age, Registration_Number, Convocation_Registration_Number)
    VALUES (p_last_name, p_address, p_age, p_registration_number, p_convocation_reg_number);

    -- Update Student table with convocation registration number
    UPDATE Student
    SET Convocation_Registration_Number = p_convocation_reg_number
    WHERE Registration_Number = p_registration_number;
END//
DELIMITER ;
```

4. Perform the registration for Akila, Haritha, Saadia, Thamish and Ridma with convocation registration numbers 1, 2, 3, 4 and 5 respectively. Give suitable meaningful values for the rest of the columns for each of these students. *Observe the updates made to the Student table.*

SQL File 3*

Limit to 1000 rows

```
-- Execute the stored procedure for each student
CALL RegisterStudent('Akila', '123 Main St, Colombo', 23, 154, 1);
CALL RegisterStudent('Haritha', '456 Lake Rd, Kandy', 22, 118, 2);
CALL RegisterStudent('Saadia', '789 Hill St, Galle', 24, 147, 3);
CALL RegisterStudent('Thamish', '321 Beach Ave, Matara', 23, 379, 4);
CALL RegisterStudent('Ridma', '654 Garden Rd, Negombo', 22, 155, 5);

-- Check the updates made to Student table
SELECT * FROM Student WHERE Convocation_Registration_Number IS NOT NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conte

Name	Registration_Number	GPA	Convocation_Registration_Number	Class
Haritha	118	3.75	2	NULL
Saadia	147	4.00	3	NULL
Akila	154	3.80	1	NULL
Ridma	155	3.85	5	NULL
Thamish	379	3.95	4	NULL

5. Create a view called Registered to retrieve the details Name, Registration Number, GPA, Convocation Registration Number, Address and Age of those students who have registered for the convocation. Observe the view with the respective data.

SQL File 3*

```sql
-- task 5
CREATE VIEW Registered AS
SELECT
    s.Name,
    s.Registration_Number,
    s.GPA,
    s.Convocation_Registration_Number,
    c.Address,
    c.Age
FROM Student s
INNER JOIN Convocation c ON s.Registration_Number = c.Registration_Number
WHERE s.Convocation_Registration_Number IS NOT NULL;

-- View the registered students
SELECT * FROM Registered;
```

Result Grid

Name	Registration_Number	GPA	Convocation_Registration_Number	Address	Age
Akila	154	3.80	1	123 Main St, Colombo	23
Haritha	118	3.75	2	456 Lake Rd, Kandy	22
Saadia	147	4.00	3	789 Hill St, Galle	24
Thamish	379	3.95	4	321 Beach Ave, Matara	23
Ridma	155	3.85	5	654 Garden Rd, Negombo	22

6. Create a view called NotRegistered to retrieve the details Name, Registration Number, GPA, Address and Age of those students who have not registered for the convocation. Observe the view with the respective data.

SQL File 3*

```sql
CREATE VIEW NotRegistered AS
SELECT
    s.Name,
    s.Registration_Number,
    s.GPA,
    'Not Available' AS Address,
    0 AS Age
FROM Student s
WHERE s.Convocation_Registration_Number IS NULL;

-- View the non-registered students
SELECT * FROM NotRegistered;
```

Result Grid

Name	Registration_Number	GPA	Address	Age
Ishta	149	3.55	Not Available	0
Ravindu	224	3.10	Not Available	0
Chamara	327	3.25	Not Available	0
Vilakshan	373	3.70	Not Available	0
Nimuthu	398	3.25	Not Available	0

7. Create a table called LateRegistration with the same columns as the Convocation table to store the details about the students who do the late registration.

```sql
-- task 7
CREATE TABLE LateRegistration (
    Last_Name VARCHAR(50) NOT NULL,
    Address VARCHAR(100) NOT NULL,
    Age INT NOT NULL,
    Registration_Number INT NOT NULL,
    Convocation_Registration_Number INT PRIMARY KEY,
    FOREIGN KEY (Registration_Number) REFERENCES Student(Registration_Number)
);
```

8. Write a trigger to monitor the state of the LateRegistration table, so that after a late registration is done in this table, the Student table should be updated with the relevant Convocation Registration Number for that student.

```sql
-- task 8
DELIMITER //
CREATE TRIGGER UpdateStudentAfterLateRegistration
AFTER INSERT ON LateRegistration
FOR EACH ROW
BEGIN
    UPDATE Student
    SET Convocation_Registration_Number = NEW.Convocation_Registration_Number
    WHERE Registration_Number = NEW.Registration_Number;
END//
DELIMITER ;
```

9. Perform the registration as late registration for Ishta, Chamara, Nimuthu, Ravindu and Vilakshan with registration numbers 6, 7, 8, 9 and 10 respectively. Give suitable meaningful values for the rest of the columns for each of these students. *Observe the updates made to the Student table.*

SQL File 3*

```sql
-- task 9

INSERT INTO LateRegistration VALUES
('Ishta', '111 Temple Rd, Anuradhapura', 23, 149, 6),
('Chamara', '222 Market St, Kurunegala', 24, 327, 7),
('Nimuthu', '333 School Lane, Ratnapura', 22, 398, 8),
('Ravindu', '444 Station Rd, Badulla', 25, 224, 9),
('Vilakshan', '555 Park Ave, Jaffna', 23, 373, 10);

-- Check the updates made to Student table
SELECT * FROM Student ORDER BY Registration_Number;
```

Result Grid

Name	Registration_Number	GPA	Convocation_Registration_Number	Class
Haritha	118	3.75	2	NULL
Saadia	147	4.00	3	NULL
Ishta	149	3.55	6	NULL
Akila	154	3.80	1	NULL
Ridma	155	3.85	5	NULL
Ravindu	224	3.10	9	NULL
Chamara	327	3.25	7	NULL
Vilakshan	373	3.70	10	NULL
Thamish	379	3.95	4	NULL
Nimuthu	398	3.25	8	NULL

10. Create a stored procedure to and fill the class column as follows.

    a. GPA >=3.7 First class honors
    b. GPA <3.7 and GPA>=3.3 Second class honors-upper division
    c. GPA <3.3 and GPA>=2.7 Second class honors-lower division
    d. GPA <2.7 and GPA>=2.0 Third class honors

Call the stored procedure which you created at this step and observe the class values assigned to each student.

SQL File 3*

```sql
-- task 10
DELIMITER //
CREATE PROCEDURE AssignClasses()
BEGIN
    -- Update all students' classes based on their GPA
    UPDATE Student
    SET Class = CASE
        WHEN GPA >= 3.7 THEN 'First class honors'
        WHEN GPA >= 3.3 AND GPA < 3.7 THEN 'Second class honors-upper division'
        WHEN GPA >= 2.7 AND GPA < 3.3 THEN 'Second class honors-lower division'
        WHEN GPA >= 2.0 AND GPA < 2.7 THEN 'Third class honors'
        ELSE 'No honors'
    END;
END//
DELIMITER ;

-- Call the stored procedure
CALL AssignClasses();
```

Query 1

Limit to 1000 rows

```
1 •  SELECT * FROM Student;
```

Result Grid | Filter Rows: | Edit: | Export/Import

	Name	RegNo	GPA	ConvRegNo	Class
▶	Haritha	118	3.75	2	First class honors
	Saadia	147	4.00	3	First class honors
	Ishta	149	3.55	6	Second class honors-upper division
	Akila	154	3.80	1	First class honors
	Ridma	155	3.85	5	First class honors
	Ravindu	224	3.10	9	Second class honors-lower division
	Chamara	327	3.25	7	Second class honors-lower division
	Vilakshan	373	3.70	10	First class honors
	Thamish	379	3.95	4	First class honors
	Nimuthu	398	3.25	8	Second class honors-lower division